

Chapter 7

Multiplexer

This chapter introduces a switch matrix device with its possible usecases in an experimental setup. Additionally, technical data regarding hardware design and performance metrics are provided.

7.1 Introduction

Pre characterisation is an essential step in the early stages of investigating an experimental sample. Valuable information regarding various aspects of the sample, ranging from for e.g. fabrication failure to bonding errors, can be extracted during pre-characterisation. As device designs become more and more complex, the possibilities of different pre-characterisation measurements also increase rapidly. Alternatively, high throughput fabrication demands fast and efficient methods of characterising sample performance. Experimental setups are limited in resources and manually changing the cabling for large number of measurements can be a significant bottleneck. Commercial solutions are often hard to integrate in ever changing experimental stations and are not adaptable enough.

We developed a general purpose switch matrix¹ to multiplex resources (for e.g. voltage sources, measurement instruments, etc.) in an experimental setup to aid fast pre-characterisation of samples with minimal manual intervention.

¹Affectionately named Muxi.

7.2 Hardware Design

The switch matrix can route 96 input lines to 4 output channels. This configuration offers enough input lines for most experimental setups and can be used to carry out four-point measurements. The device is controlled and interfaced via Arduino Due and the multiplexer is designed as a plug-on shield (Figure 7.3) for it. 96:4 multiplexing is realised by connecting together $12 \times 16:2$ ICs which are interfaced using the I2C protocol. Table 7.1 lists the principal hardware components.

A communication link is established to the multiplexer via a serial interface. The input side is accessible through a high density 100 pin D-Sub connector on the top of the multiplexer while the 4 output channels are connected to standard female BNC connectors. Figure 7.2 depicts the various connection possibilities realizable with the multiplexer. We emphasize, that unlike usual multiplexers that only route a single input to a single output, the present device allows *many-to-one* switching. Native 12 bit ADC and DAC of Arduino-Due are provided for coarse measurements without external measurement equipment. Attention must be paid while using the ADC and DAC capabilities, as they have low resolution, limited range and might require calibration for each board.

Figure B.1 shows the detailed schematic of the multiplexer. Figure 7.1 gives a simplified functional diagram of the switch matrix and figure 7.2 explains various switching possibilities.

Details about the SCPI and QCoDes interface of the multiplexer are given in appendix B.

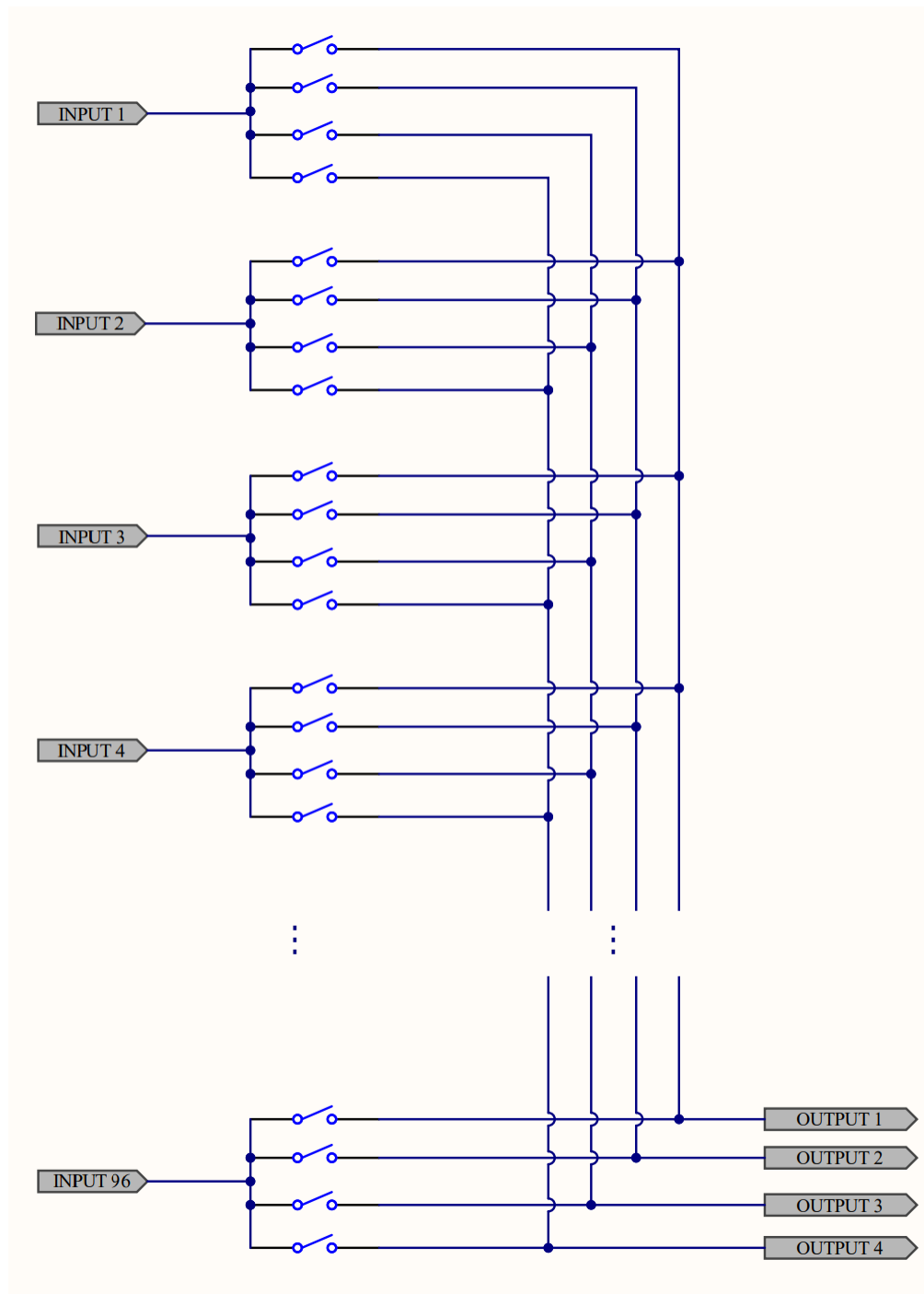


Figure 7.1: Simplified functional diagram of the multiplexer. Each of the 96×4 switches is independently controllable.

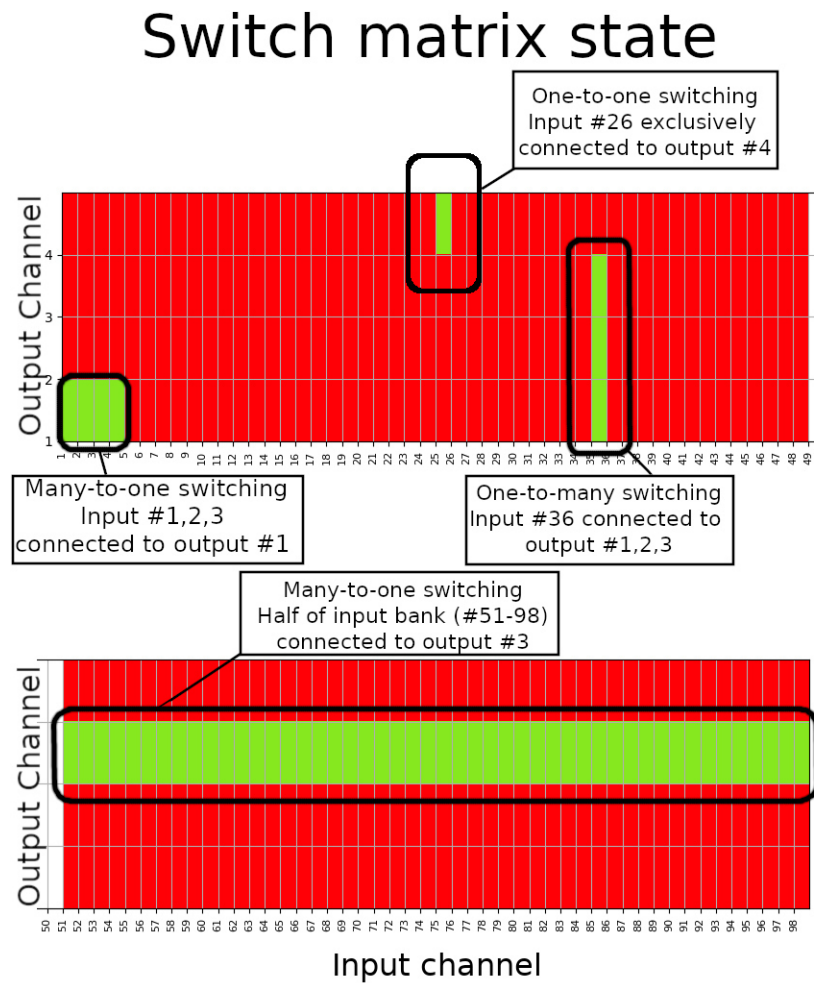


Figure 7.2: Simplified functional diagram of the multiplexer. Each of the 96×4 switches is independently controllable.

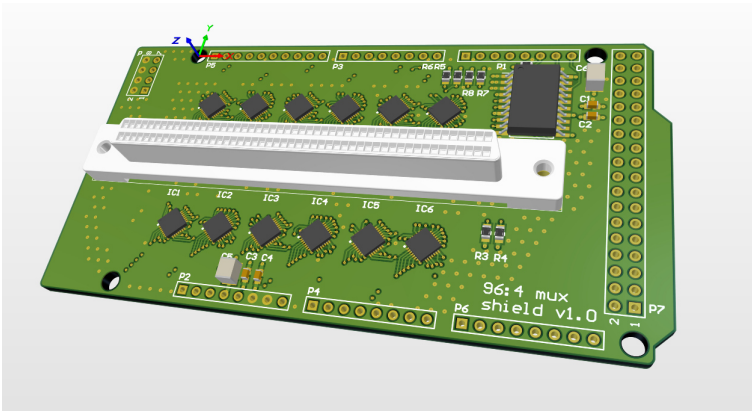


Figure 7.3: 3D model of the multiplexing shield

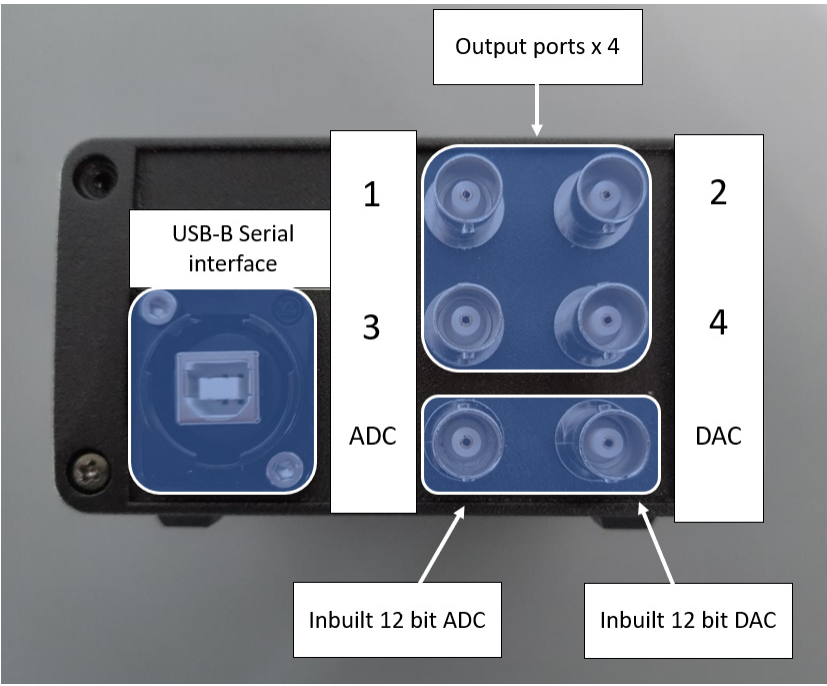
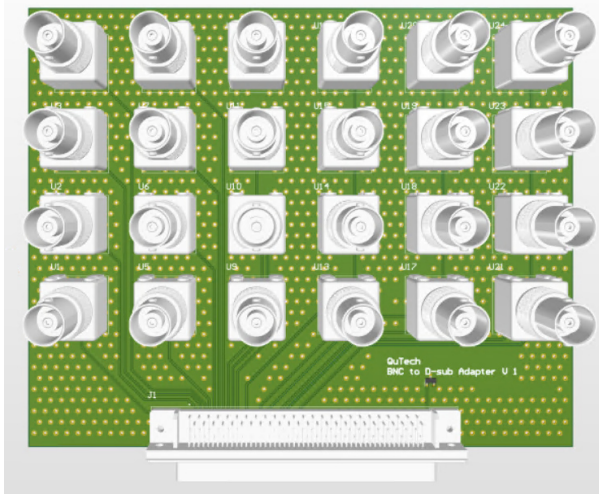


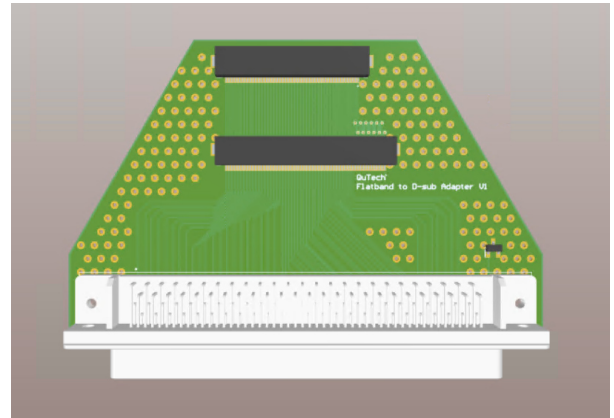
Figure 7.4: Multiplexer: front view



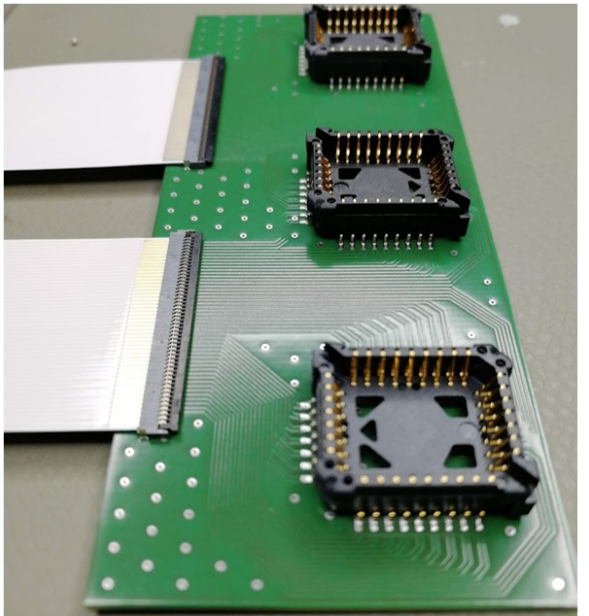
Figure 7.5: Multiplexer: side view



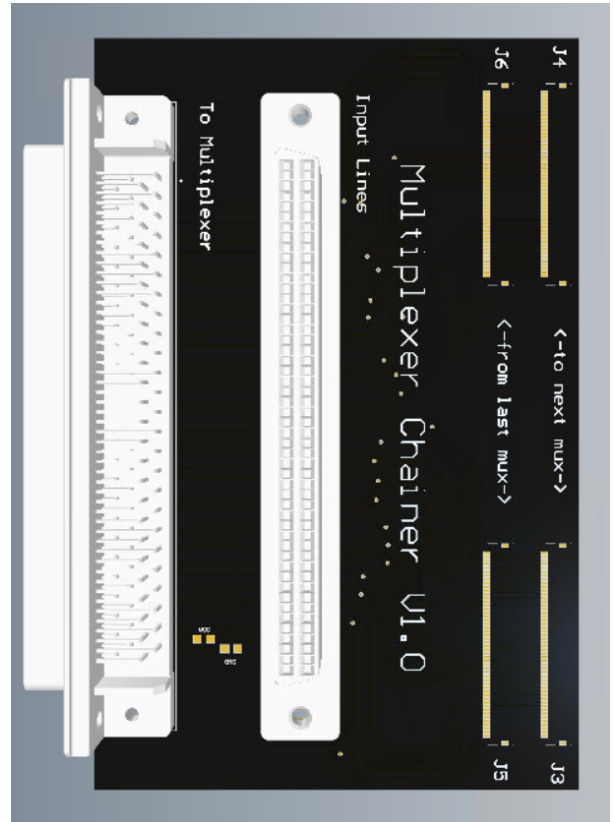
(a) 24 BNC to D-Sub



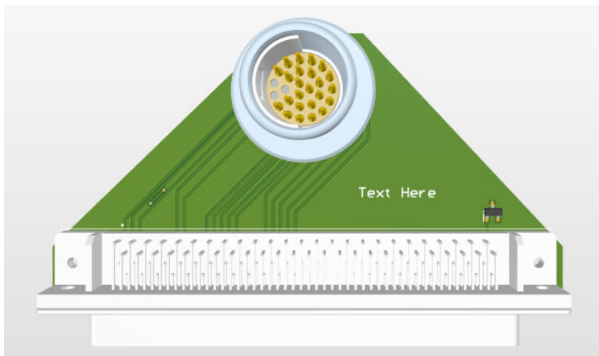
(b) 50/60 pad flatband to D-Sub



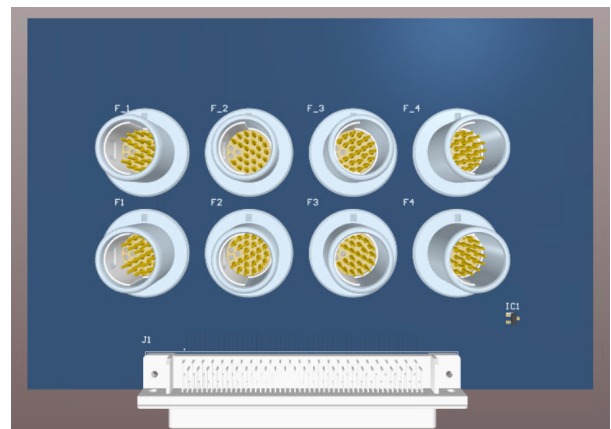
(c) 3 chip carriers to D-Sub



(d) Flatband adapter to connect multiple multiplexer together



(e) 1 × 24 pin Fischer connector to D-Sub



(f) 4 × 24 pin Fischer connector to D-Sub

Figure 7.6: Various adapters for connecting to the multiplexer

Name	Manufacturer	Description
Arduino Due	Arduino	Microcontroller board for serial communication and chip control
MAX14661ETI+T Integrated Circuit	Maxim Integrated	16:2 multiplexer chip with an I2C interface
PCA9544ADWR Integrated Circuit	Texas Instruments	Chip to multiplex I2C communication lines to connect large number of slave ICs with limited addresses
5749070-9 D-Sub Connector	TE Connectivity	100 pin female D-Sub connector for input connections
Standard BNC Female Connector	-	Standard female BNC connectors for output and DAC/ADC connections
Standard USB-B Female Connector	-	Standard female USB-B connector for serial interface and power

Table 7.1: Main hardware components of the switch matrix

7.3 Adapters

The choice of a high density D-Sub connector on the input side of the multiplexer is chiefly guided by the large number of pins available in it. This connector is uncommon and therefore we provide various adapters that plug into the input side for connecting to the multiplexer² as shown in figure 7.6.

- **BNC $\times$ 24** : 24 standard BNC connectors to D-Sub
- **Fischer** : 1 and 4 24 pin Fischer connectors to D-Sub
- **Flatband**: 50 and 60 pads flatband connectors to D-Sub
- **Chip carrier** : 3 chip carriers to D-Sub (designed by Lino Visser[25])
- **Chainer** : adapter to connect multiple multiplexers together using flatband cables to effectively increase number of output channels per multiplexer

7.4 Applications

In this section, we outline three applications of the multiplexer aimed towards aiding in pre-characterisation and testing samples and setups quickly.

7.4.1 Application I: Resource Sharing

The simplest application of the switching matrix is to share limited number of resources available in an experiment station. Various sample terminals that need to be biased at the same level can be biased by utilising only a single DAC channel and multiplexing it. Similarly, slow dynamic signals can also be multiplexed (See section 7.5.4 for AC performance of the multiplexer). Essentially, different combinations of sample terminals can be connected together via the multiplexer without taking the sample outside of a cryostat or manually changing the cabling. The multiplexer has been extensively used in [25], for mass characterisation of industrially fabricated samples.

²CAD files for the multiplexer and various adapters can be found at the group's altium-ecad repository.

7.4.2 Application II: Leakage Matrix

Fast and reliable pre-characterisation of a sample before carrying out advanced measurements is a crucial task, as sample failure can be detected early on. Imaging methods such as optical microscopy or scanning electron microscopy are often not suitable for exhaustive investigation of samples with complex designs. Here we propose an application of the switching matrix to study the leakage behaviour between different terminals of a sample. Figure 7.7a shows the circuit for such a measurement. An N -terminal **DUT** is connected to the input of the multiplexer, which routes a pair of DUT terminals to two multiplexer outputs. One output channel is biased by a DAC and the other one is connected to an ammeter which measures the current to ground. The DAC voltage is swept while measuring the leakage current. With help of the multiplexer, all possible $\sim N^2$ pairs of terminals are swept sequentially.

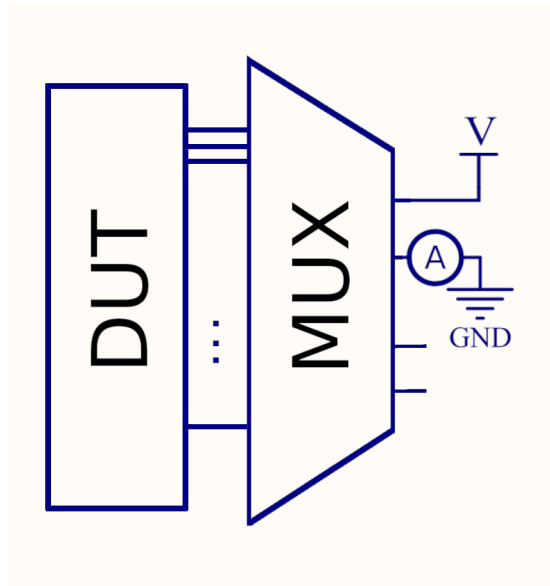
We obtain an $N \times N$ matrix³ of leakage I-V sweeps using the method described above, from which important information can be obtained about device behaviour. The leakage matrices are interactive, in the sense, the user can click on a grid point and open the corresponding leakage sweep. Thus, making sample debugging intuitive. Moreover, It is possible to speed up leakage matrix measurements by utilising all four output channels of the multiplexer and using buffered readout to eliminate communication overhead during measurements.

Figure 7.8a shows an example of maximum leakage current extracted from a leakage matrix for a 15 terminal sample from Infineon AG at 20mK. The red regions correspond to a leakage current of more than $I_{\text{threshold}} = 1\text{nA}$. Large sectors of leaky paths are clearly noticeable.

Leakage matrix measurement can be used to not only find out leakage current levels but also to identify the type of leakage. We can pre-process the data⁴ to highlight certain features and then cluster similar sweeps automatically using standard clustering algorithms such as the K-means algorithm[1]. In practise, scaling the data between a unit interval helps in clustering similar shapes of curves and applying a log transform helps in clustering together sweeps with similar leakage current levels. Identifying these clusters can aid in finding out device failure modes and give feedback in a fabrication loop. Figures 7.10 and 7.9 show the leakage matrix of the Infineon sample clustered into three sectors. We do observe clustering of similar shapes and leakage levels at a crude level, but various curves are mislabelled. Automatic clustering of leakage levels without human intervention can be

³Diagonal terms corresponding to a terminal being tested against itself do not constitute a real measurement. Only $N^2 - N$ sweeps are carried out.

⁴Each I-V sweep corresponds to one data point.



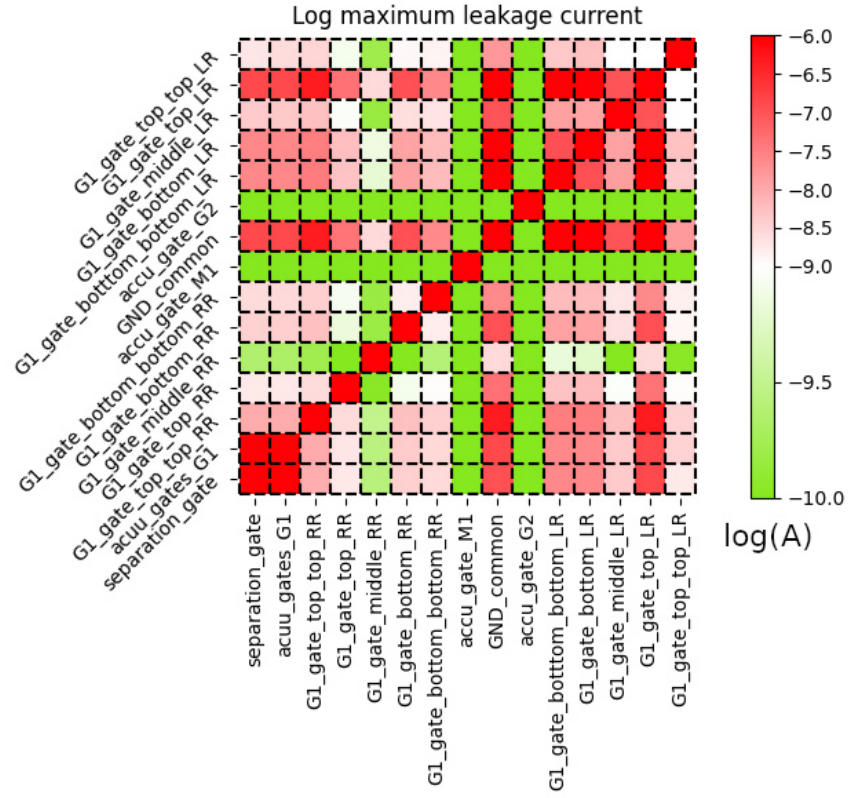
(a) Circuit for measuring leakage matrices

extremely convenient for characterising devices at a mass scale. Performance of clustering algorithms is also expected to significantly improve for large datasets of I-V sweeps.

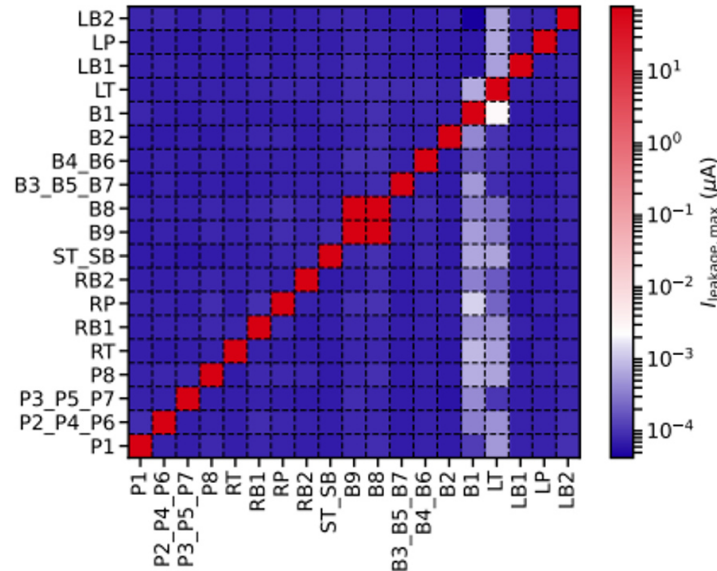
7.4.3 Application III: Line testing

Troubleshooting faulty contacts in an experimental station is a routine task. These could be DC lines going into a cryostat, traces on a sample PCB, soldered pads on a high density connector, etc. While the testing mostly requires a simple continuity check, the number of combinations to check could be large. The task is simple but extremely slow and often requires more than one person. A simple broken connection can render an experiment totally inoperable and can waste valuable fridge time. The frustration is even worse!

We can take advantage of the multiplexer's inbuilt ADC and DAC along with its fast switching capabilities to speed up line testing significantly. Circuit diagram for the line probing arrangement is shown in figure 7.11a. The contacts to which leakage or a short is suspected are connected to the input side of the multiplexer. A probe is placed on a contact that needs to be investigated. The multiplexer sequentially connects all inputs to ground. Resistance between the probe and ground terminal is measured using the internal

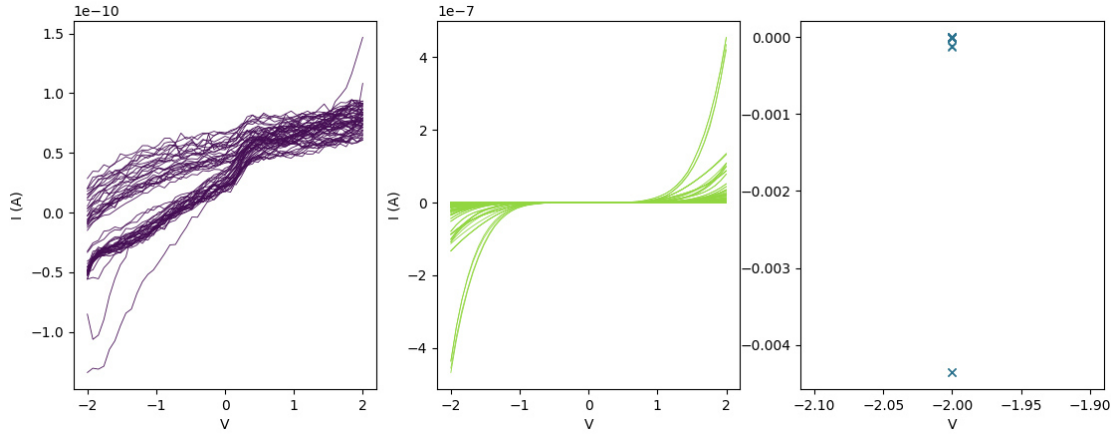


(a) Leakage current matrix for a 15 terminal double dot device from Infineon AG

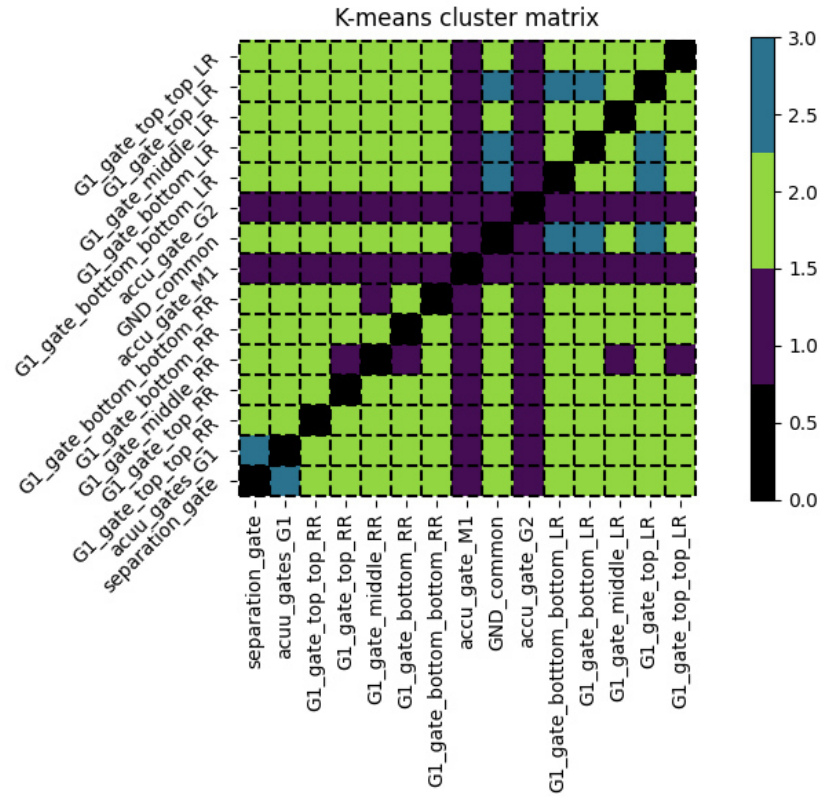


(b) Leakage current matrix for an 8 dot device. (by Lino Visser [25])

Figure 7.8: Examples of leakage matrices with large leakage currents

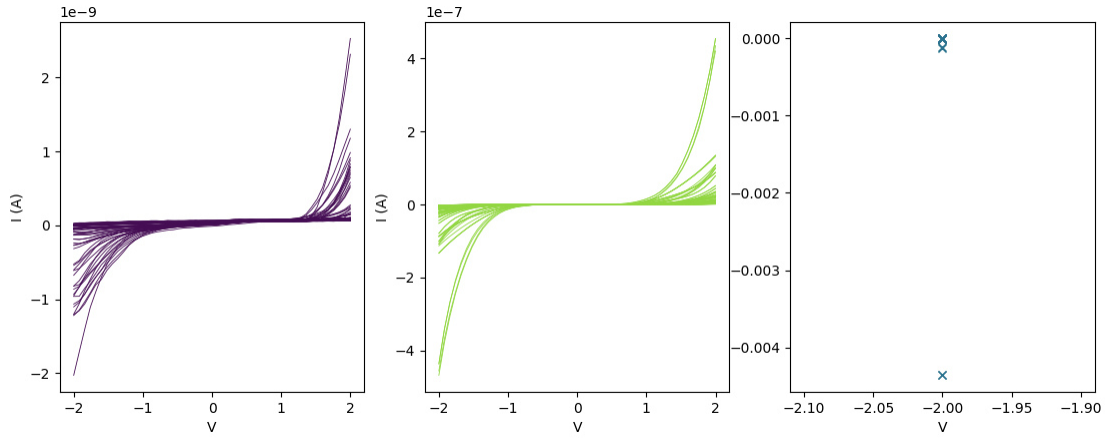


(a) Three clusters separating the I-V sweeps according to their shape. Left: Low magnitude leakage curves resembling shape of self leakage of multiplexer. Middle : High magnitude diode like leakage curves. Right: Extremely large leakage currents ($> 1\mu\text{A}$) resulting in sweep abort.

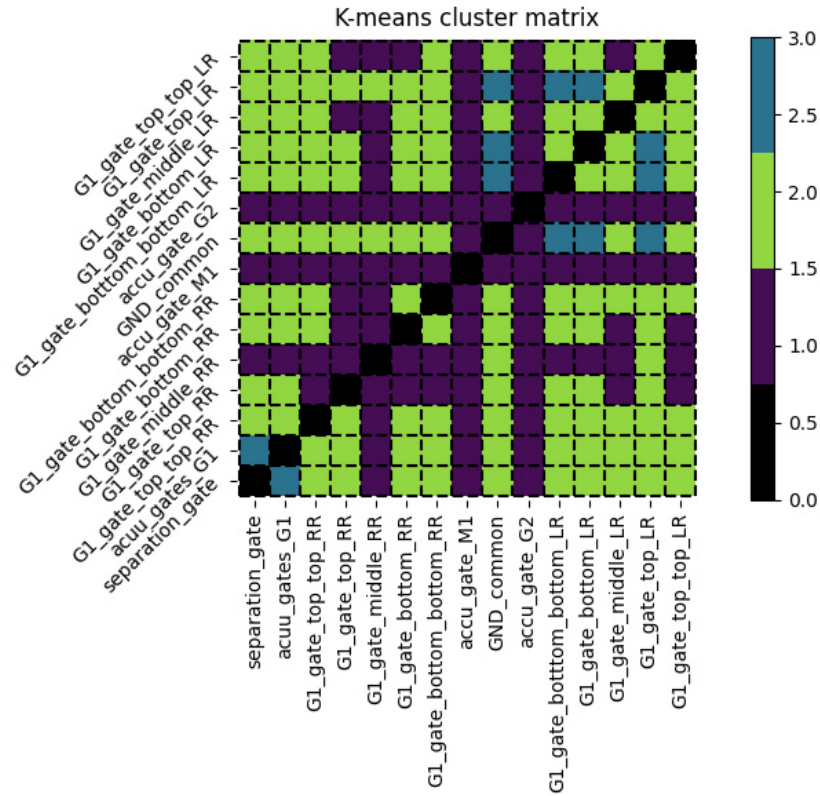


(b) Matrix representation of the above clusters. Each color corresponds to a cluster.

Figure 7.9: Leakage matrix clustering according to the shape of leakage current curve



(a) Three clusters separating the I-V sweeps according to their magnitude. Left: nA scale leakage curves. Middle : Large leakage currents $\sim 100\text{nA}$ Right: Extremely large leakage currents ($> 1\mu\text{A}$) resulting in sweep abort.



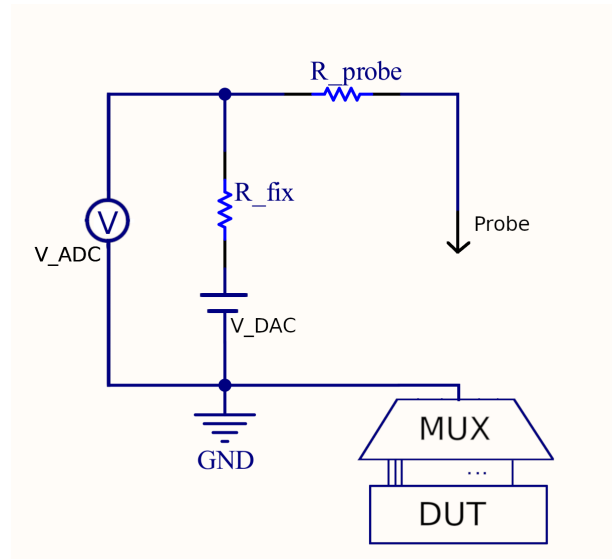
(b) Matrix representation of the above clusters. Each color corresponds to a cluster.

Figure 7.10: Leakage matrix clustering according to leakage current magnitude

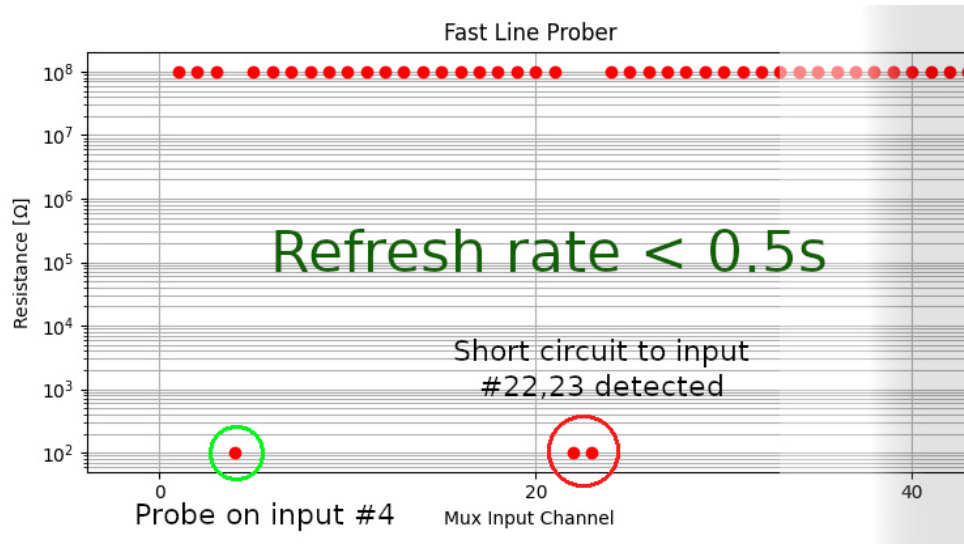
ADC and DAC.

$$R + R_{\text{Probe}} = R_{\text{fix}} \left[\frac{V_{\text{DAC}}}{V_{\text{ADC}}} - 1 \right]^{-1}$$

This is repeated routinely and we get resistance between the probe and all the multiplexer inputs at a refresh rate of less than half a second. An illustrative example of the line probe is shown in [7.11b](#). The fast refresh rate of $\leq 0.5\text{s}$ is achieved by sending resistance data to PC for the whole multiplexer in a single shot, thus minimising communication overhead.



(a) Circuit for testing fridge/pcb lines quickly.



(b) Line probe placeholder.

Figure 7.11: Fast line prober circuit and example.

7.5 Technical Specifications

After providing a few usecases in the last section, we provide data about the hardware performance of the multiplexer here. Detailed performance data about the ICs used can be found in [22] and [14].

7.5.1 Current and Voltage Ratings

Voltage signals between -5.5V to $+5.5\text{V}$ are permissible at the input and output terminals. A maximum current of 50nA can safely flow through the switches[14].

7.5.2 Switching Time

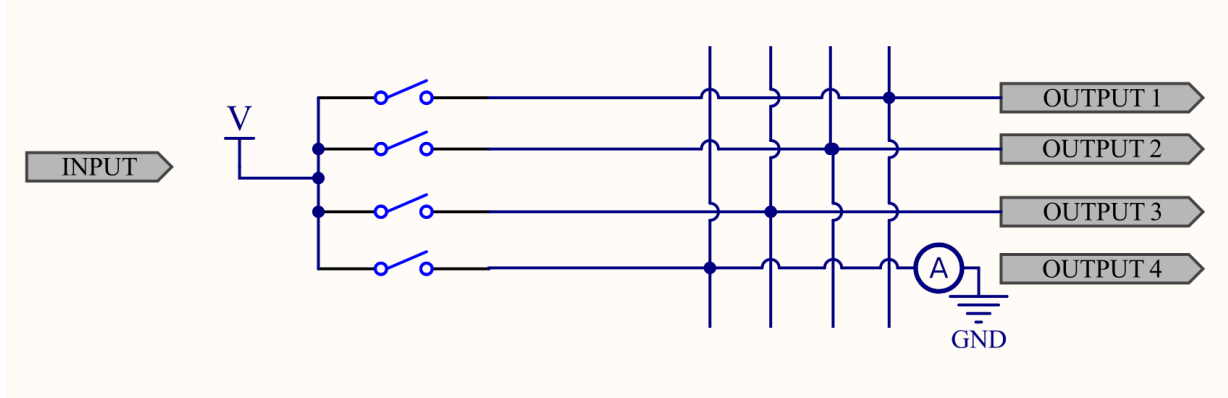
Table 7.2 lists the time taken for various operations of the multiplexer. Most operations take less than 10ms but the slowest ones can take upto $\sim 35\text{ms}$. Measurements scripts involving the multiplexer should be written accordingly. The multiplexing chips require only a few μs for switching operations. The switching times in table 7.2 are limited by speed of serial communication with the Arduino Due. The biggest overhead is serial communication as can be seen from many different operations requiring the same time $\sim 4\text{ms}$.

7.5.3 Leakage Current

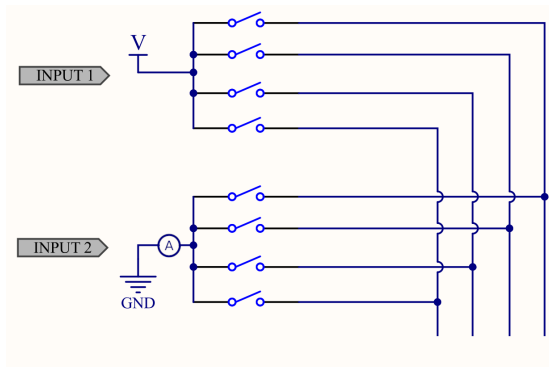
The self leakage currents between input and output channels of the multiplexer are plotted in figures 7.13a, 7.13b, and 7.14a and corresponding test circuits are shown in figure 7.12. Leakage current of upto 100pA is observed, making the multiplexer unsuitable for very sensitive measurements.

7.5.4 AC Response

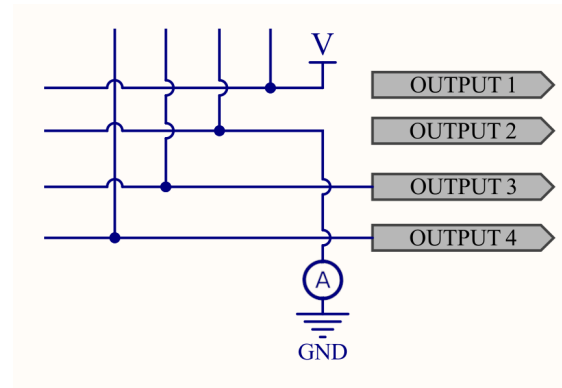
Figure 7.15 shows the attenuation and phase difference introduced by the multiplexer between its input and output port. The signal amplitude drops to about 75% and a phase lag of upto 50° is observed at 5MHz . These measurements were carried out using the Zurich Instruments MFLI.



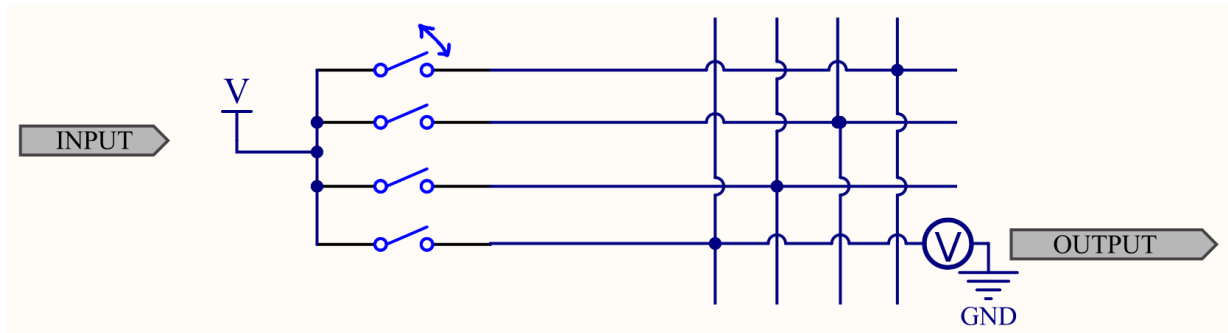
(a) Test circuit for leakage current and AC response measurement between input and output terminals.



(b) Test circuit for leakage current measurement between two input terminals.



(c) Test circuit for leakage current measurement between two output terminals.



(d) Test circuit to create an artificial square pulse at the multiplexer output to study (dis)-charging effects.

Figure 7.12: Test circuits for multiplexer characterisation.

Operation	Time	Description
Close switch	$4.1 \pm 0.4\text{ms}$	Close a single switch
Open switch	$8.2 \pm 0.4\text{ms}$	Open a single switch
Read switch	$4.1 \pm 0.2\text{ms}$	Read state of a single switch
Set output channel	$18.2 \pm 0.5\text{ms}$	Set one whole output channel
Read output channel	$34.8 \pm 1.3\text{ms}$	Read one whole output channel
Set input channel	$2.5 \pm 0.5\text{ms}$	Set one whole input channel
Read input channel	$5.7 \pm 1.7\text{ms}$	Read one whole input channel
Set DAC	$4.1 \pm 0.3\text{ms}$	Set voltage at DAC
Get DAC	$4.1 \pm 0.2\text{ms}$	Read DAC voltage
Get ADC	$4.1 \pm 0.3\text{ms}$	Read voltage from ADC
Get ADC*	$4.1 \pm 0.3\text{ms}$	Read voltage from ADC averaged over 100 samples
Reset	$4.1 \pm 0.3\text{ms}$	Reset whole switch matrix

Table 7.2: Speed of various QCoDes operations

7.5.5 Noise Levels

Figure 7.16 shows the voltage noise at the input and output ports of the multiplexer. Noise spectral density of several $\mu\text{V}/\sqrt{\text{Hz}}$ is observed at higher frequencies, which deems the multiplexer unsuitable for highly sensitive measurements. We characterise the noise at input and output in two configurations:

1. **Isolated**: all switches in the switch matrix are open.
2. **Connected**: all possible switches to the input/output port are closed.

Higher noise levels are seen for the **connected** configuration as compared to the **isolated** configuration. These measurements were carried out using the Zurich Instruments MFLI.

7.5.6 Inbuilt DAC/ADC Performance

The Arduino Due has inbuilt 12bit DAC and ADC. The DAC has an output range of 0.55V to 2.75V. The ADC can read from GND to 3.3V at 12bit resolution. Figure 7.17 shows the DAC voltage as measured by the inbuilt ADC. The response is highly linear. However, the 12 bit resolution limits their application to only coarse measurements.

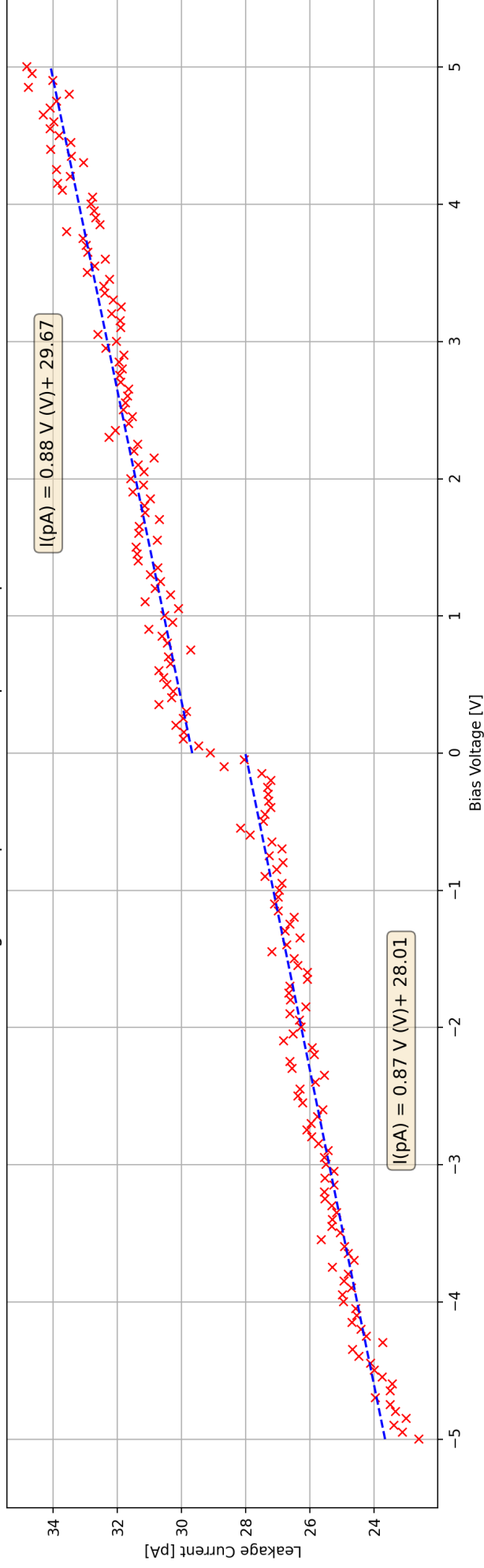
7.5.7 Charging/Discharging Effect

Figure 7.14b shows the time domain behaviour of an artificially created 100ms long square pulse through the multiplexer. The pulse is created by connecting a 1V DC signal at the input and flipping the switch between input and output at intervals of 100ms (See figure 7.12d). Table 7.3 compares various quantities of interest. The measured values quoted there, include both communication delays and charging/discharging effects. The overshoot and slow discharge is most likely a parasitic effect of the multiplexer and the connecting cables.

Quantity	Measured	Ideal	Description
Rise time	$2.6 \pm 0.2\mu s$	0	Time for rising from a low level to high level
Fall time	$1.184 \pm 0.0ms$	0	Time for falling from a high level to low level
Rising slew rate time	$306 \pm 25\mu V/ns$	∞	Rate of voltage increase from low to high level
Falling slew rate time	$-680.3 \pm 0.0nV/ns$	$-\infty$	Rate of voltage decrease from high to low level
Unit interval	$111.6 \pm 1.1ms$	100ms	Time for which the square pulse stays at high level

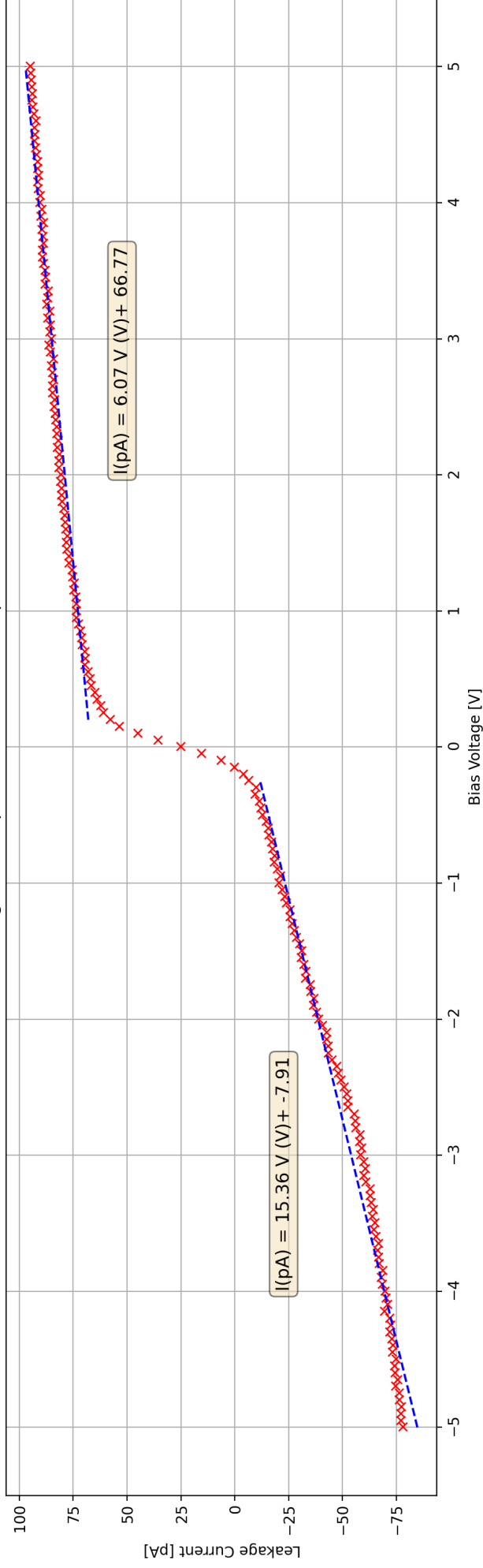
Table 7.3: Quantification of charging/discharging effects through the multiplexer

Self leakage of multiplexer between input and output terminals



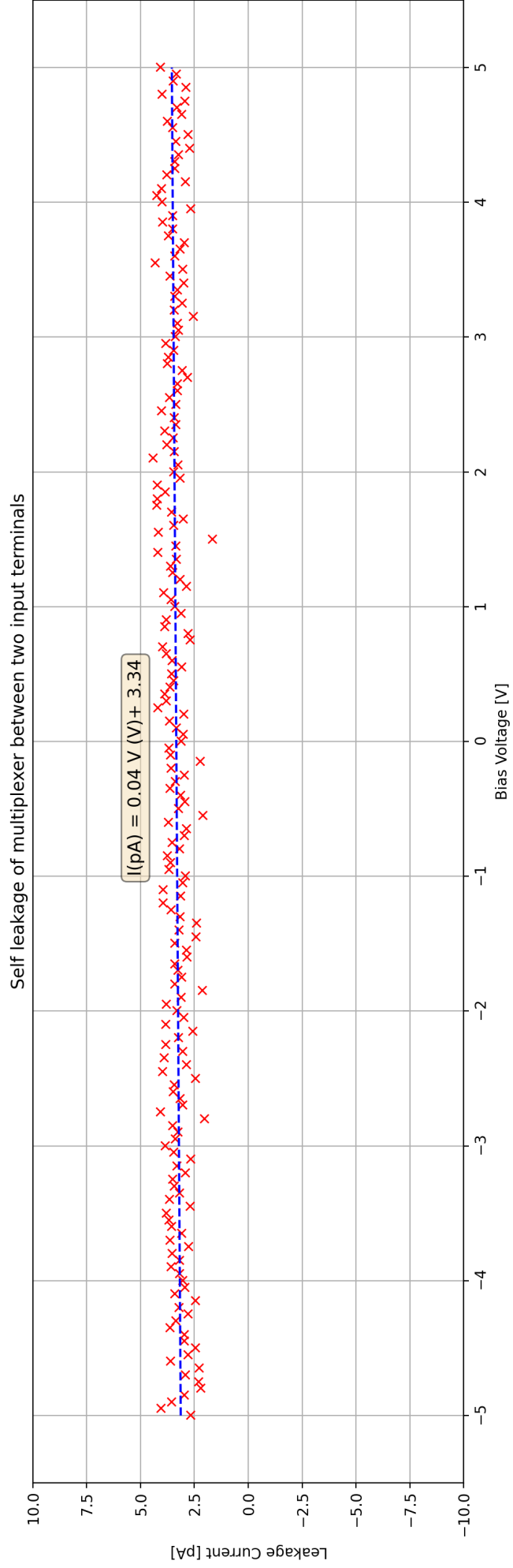
(a) Self leakage current between input and output terminals.

Self leakage of multiplexer between two output terminals

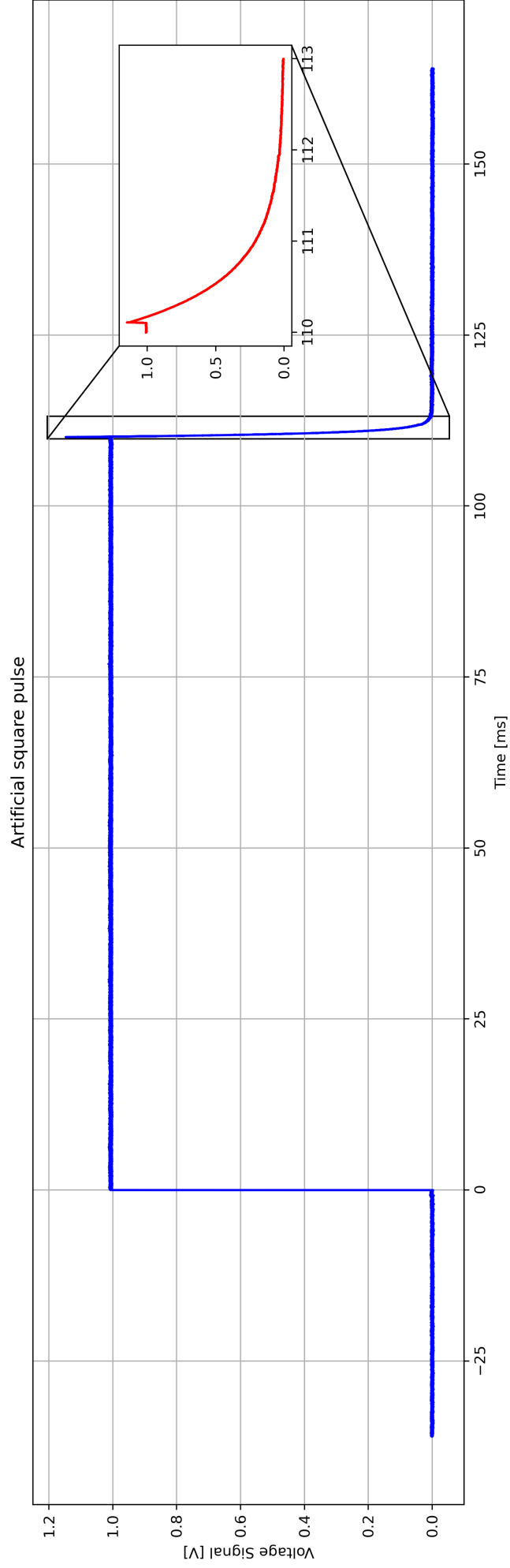


(b) Self leakage current between two output terminals.

Figure 7.13: Self leakage current between input and output terminals of the multiplexer.

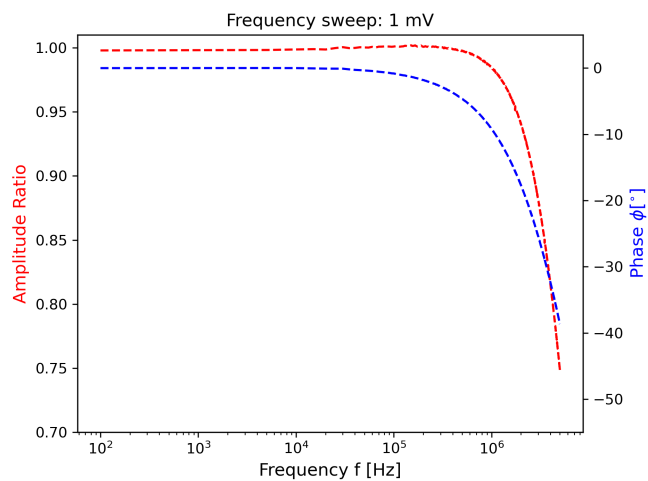


(a) Self leakage current between two input terminals.

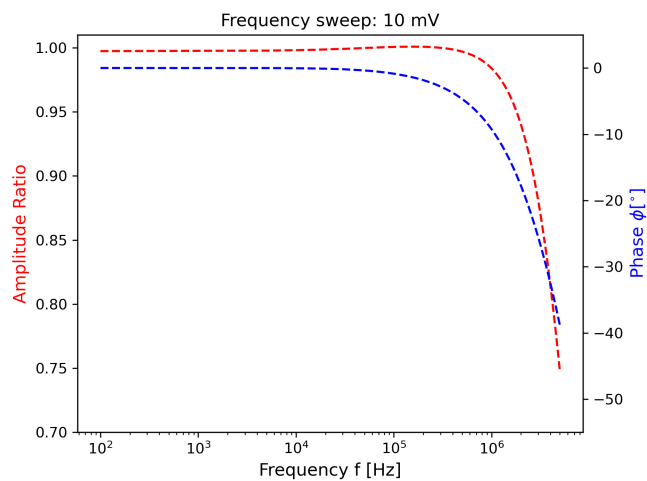


(b) Charging/discharging effect in an artificially created square pulse.

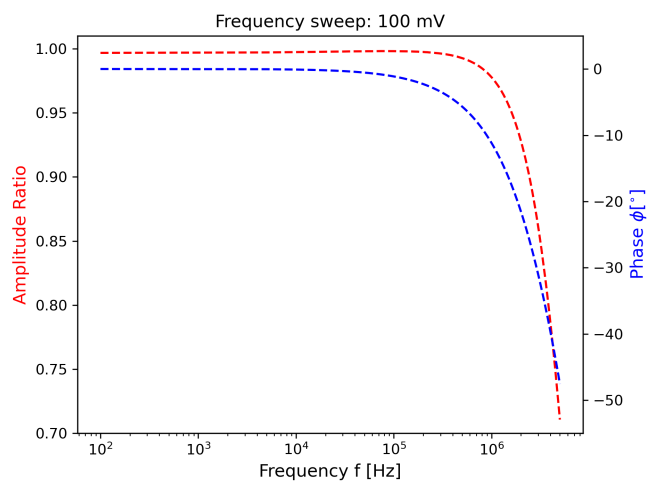
Figure 7.14: Self leakage current between input terminals and charging/discharging effect.



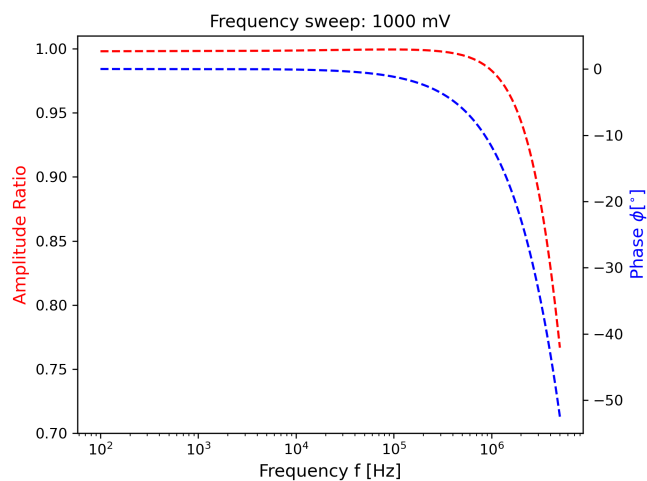
(a) Signal amplitude = 1mV



(b) Signal amplitude = 10mV

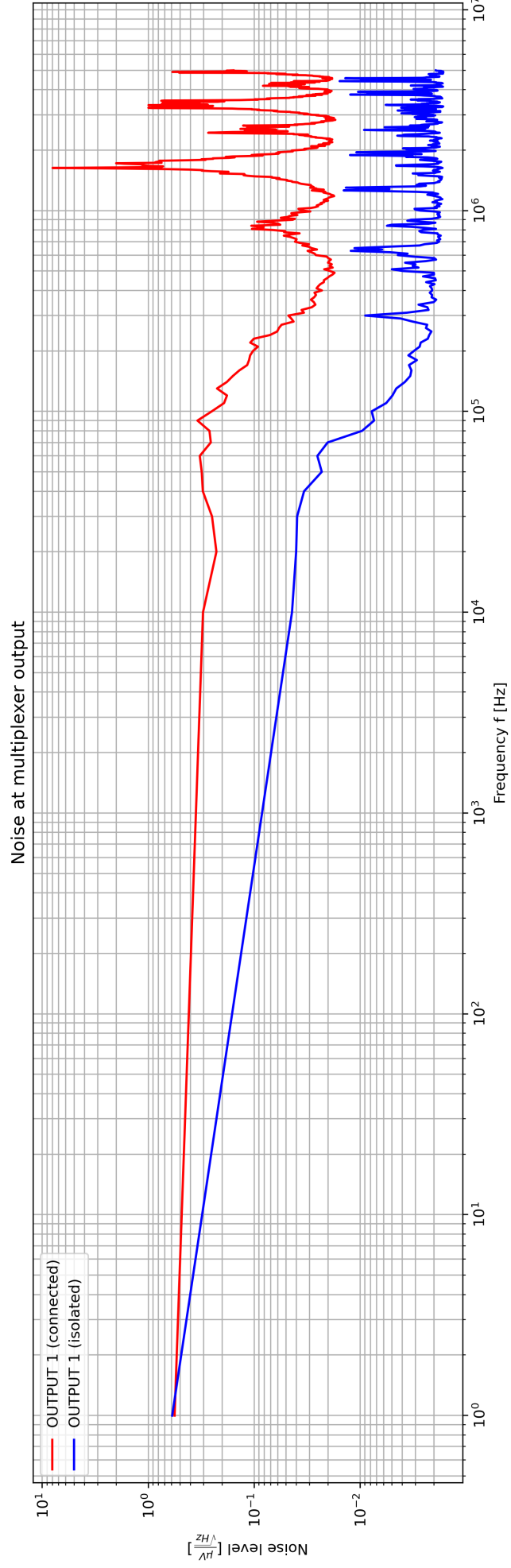


(c) Signal amplitude = 100mV

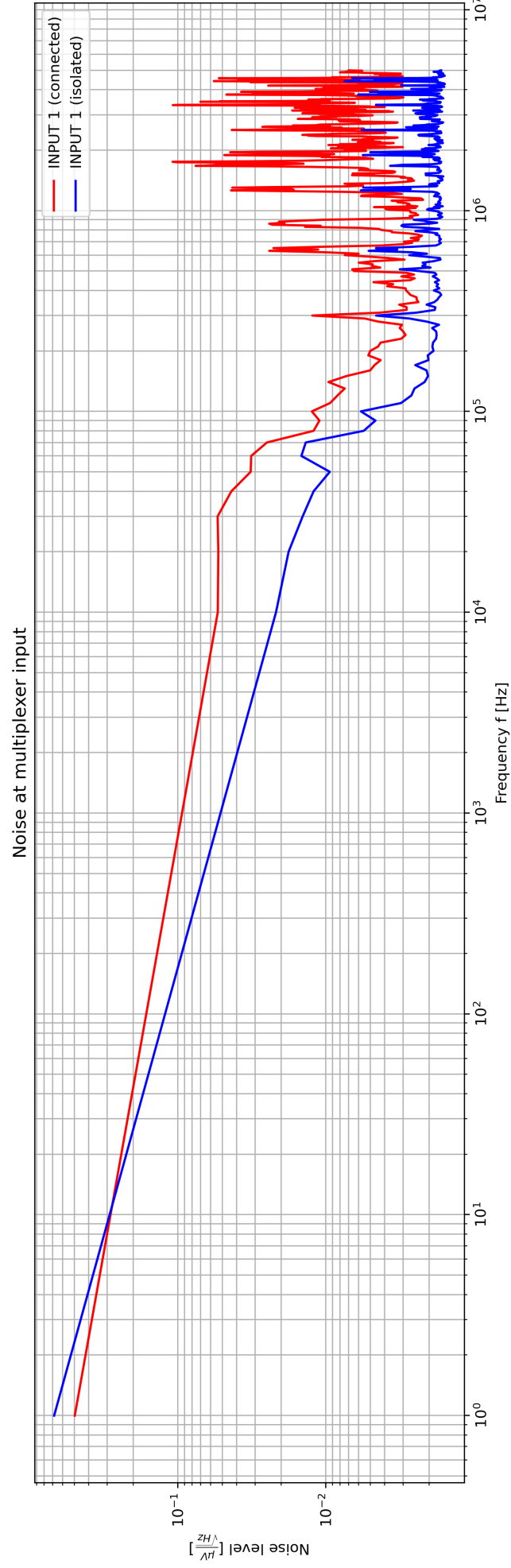


(d) Signal amplitude = 1V

Figure 7.15: Frequency response of the multiplexer according to test circuit in figure 7.12a.



(a) Noise at output terminal



(b) Noise at input terminal

Figure 7.16: Noise amplitude spectral density at input and output terminals of the multiplexer.

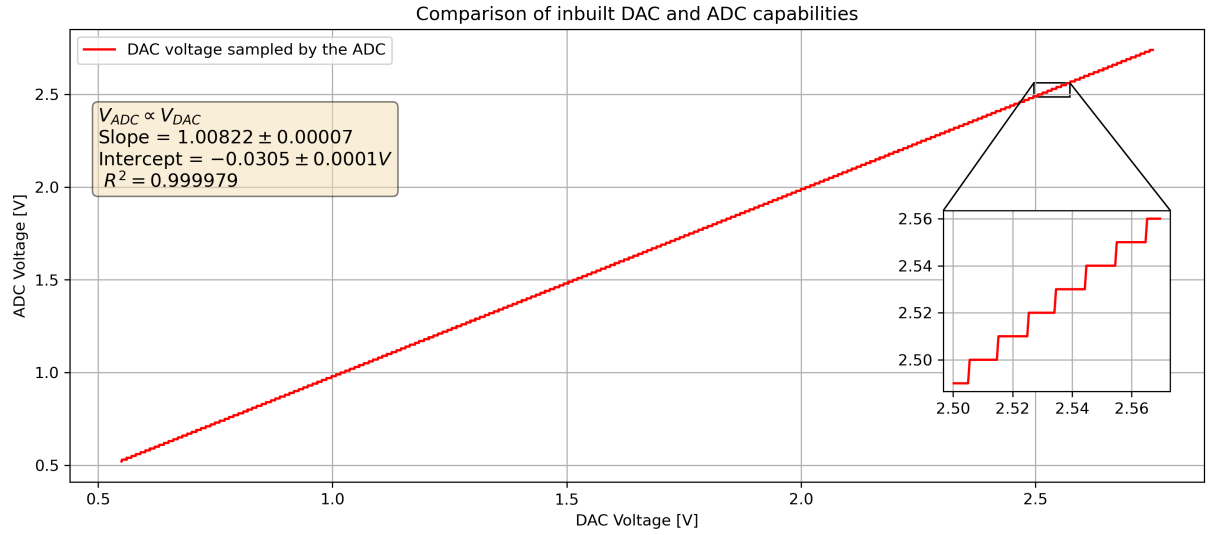


Figure 7.17: Inbuilt DAC and ADC capabilities.

Appendix B

Interfacing the Multiplexer

This appendix provides the SCPI and QCoDeS command set for interfacing the multiplexer along with some illustrative examples. The multiplexer firmware, driver, characterisation data and some usecase examples can be found at [REPOSITORY](#).

The multiplexer natively supports serial communication over its Arduino Due USB interface with

BAUD rate = 115200, read_termination = '\r\n', write_termination = '\n'

Throughout this documentation and the code, we follow the convention that the 96 inputs are labelled as PIN = 1,2,3...48 ~~49,50~~ 51,52...97,98 and 4 BNC outputs are labelled as OUT = 1,2,3,4. The seemingly unusual discontinuous numbering of the input pins arises from routing constraints and the fact that we use the numbering of the D-Sub connector here. One can access the Arduino's V_{CC} and GND at pin 49 and 50 of the D-Sub, respectively (see schematic in Fig. [B.1](#)). As this unusual labelling can be easily dealt with programmatically, we leave it as it is to avoid confusion in future hardware development related to the multiplexer.

Detailed schematic of the multiplexer can be found in figure [B.1](#). A basic command set is provided here for the multiplexer. The advanced user is advised to look into the source code present at the group's repository.

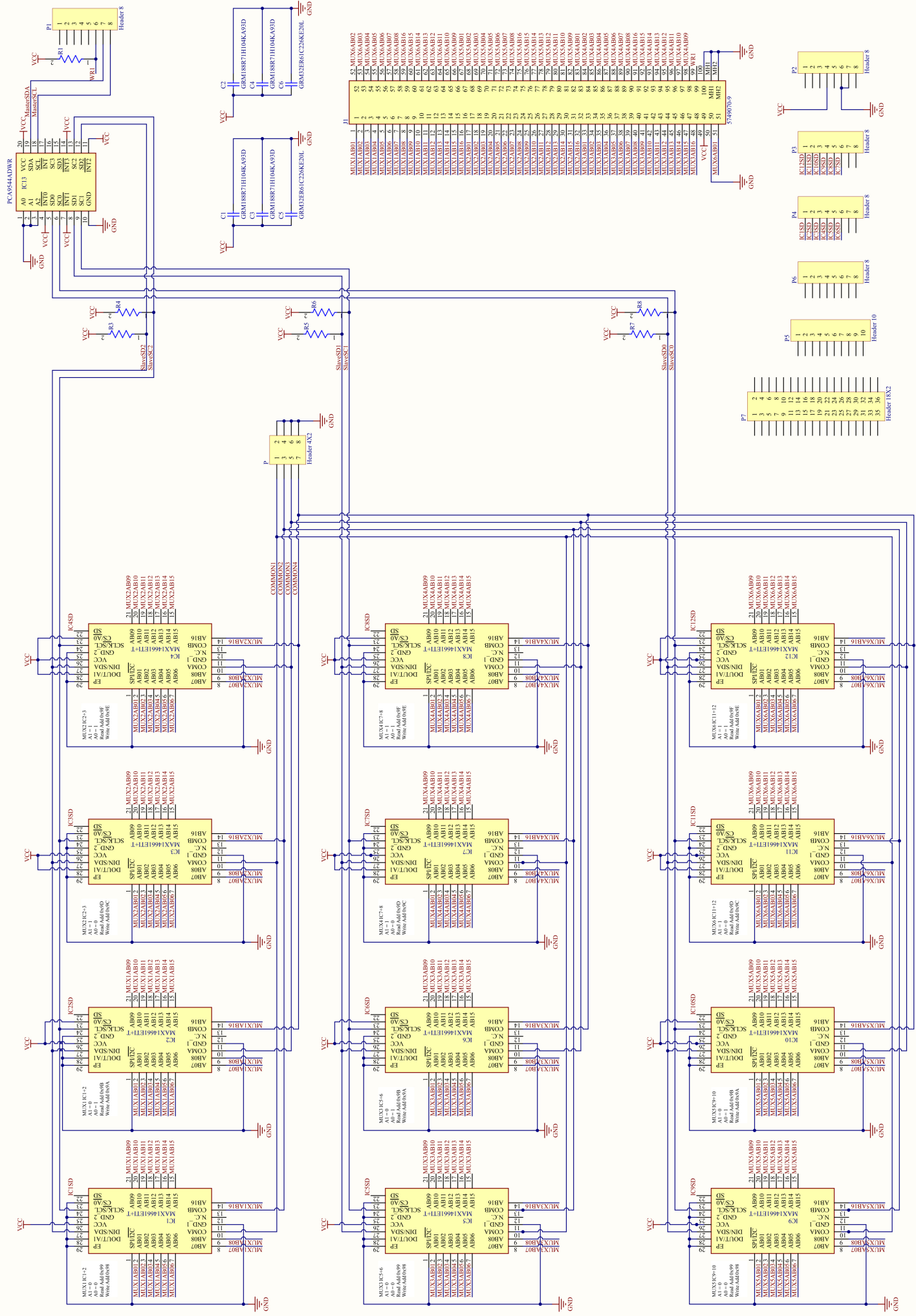


Figure B.1: Detailed schematic of the multiplexer.

B.0.1 SCPI Command Set

See Table [B.1](#)

B.0.2 QCoDeS Command Set

See Table [B.2](#)

Table B.1: SCPI Command Set

SCPI command	Parameters	Description
*IDN?	-	Identify the device
*RST	-	Reset the whole switch matrix by opening all switches
MUX:STATus?	PIN, OUT	Query the connection status between input PIN and output OUT Return values: 0 : Off (open switch); 1 : On (closed switch); 128 : Invalid inputs; 255 : failed internal I2C communication Example: MUX:STATUS? 13, 2
MUX:FULLstatus?	OUT	Query the connection status between all 96 inputs and one fixed output OUT Return value: Comma separated string of 96 switch status entries Example: MUX:FULLSTATUS? 2
MUX:SWITCh:ON	PIN, OUT	Connect input PIN to output OUT Return values: 0 : Successful connection 128 : Invalid inputs Otherwise : failed internal I2C communication Example: MUX:SWITCH:ON 13, 2

MUX:SWITCh:OFF	PIN, OUT	Disconnect input PIN to output OUT Return values: 0 : Successful disconnection 128 : Invalid inputs Otherwise : failed internal I2C communication Example: MUX:SWITCH:OFF 13, 2
DAC:SETV	Voltage [0.55V, 2.75V]	Set voltage on inbuilt 12 bit DAC Return values: 12 bit binary word written to the DAC Example: DAC:SETV 2.10
DAC:GETV?	-	Read voltage set on the inbuilt 12 bit DAC Return values: Voltage between 0.55V and 2.75V Note: This is not a measured value, instead it is the last set voltage to the DAC using SETV command. Example: DAC:GETV?
ADC:GETV?	-	Read voltage value from the 12 bit inbuilt ADC Return values: Voltage between GND and 3.30V Example: ADC:GETV?
ADC:AVERage?	N _{Samples}	Read N_{Samples} times averaged voltage value from the 12 bit inbuilt ADC Return values: Voltage between GND and 3.30V Example: ADC:AVERAGE? 100

LINEPROBE:RUN	GND V_{DAC} R_{fix}	Quickly measure resistance between fixed probe and all 96 input lines. Return values: Comma separated string of 96 resistance values between probe and input lines Example: LINEPROBE:RUN 2, 1.0, 100

Table B.2: QCoDes Parameters

Parameter		Description
Cxx_0y	xx = 01,02,...48 xx = 51,52,...98 y = 1,2,3,4	96 × 4 parameters each corresponding to the connection between input xx and output 0y Cxx_0y.get(): 0 (open switch), 1 (closed switch) Cxx_0y.set(0): Open switch Cxx_0y.set(1): Close switch
output_channel_y	y = 1,2,3,4	4 parameters each corresponding to one ouput channel channel_y.get(): list of input pins connected to output channel y channel_y.set(list of pins): connect input pins in list to output channel y without changing previous connections
input_channel_xx	xx = 01,02,...48 xx = 51,52,...98	4 parameters each corresponding to one ouput channel channel_y.get(): list of input pins connected to output channel y channel_y.set(list of pins): connect input pins in list to output channel y without changing previous connections
DAC	-	DAC.get()/.set() : read or set inbuilt DAC
ADC	-	ADC.get() : get voltage from inbuilt ADC